

ORIGAMI AS A SPATIAL AND GEOMETRIC REASONING BENCHMARK

Anonymous authors

Paper under double-blind review

ABSTRACT

Multimodal language models perform well on visual recognition and on tool use, but these abilities do not transfer directly to spatial and geometric reasoning. Geometric structure does not reduce to text or to a single image: exact angles and incidences, which surface lies in front of which, and the order in which operations were applied are not recoverable from a semantic description of a scene. We introduce **CP2Seq**, a benchmark for evaluating geometric reasoning and sequential planning in multimodal language models through origami. The benchmark contains 600 procedurally generated samples, each requiring a model to produce a valid sequence of folds from a crease pattern and a target folded state. Samples are produced by a deterministic folding engine. Ground truth is recorded as each fold happens rather than annotated afterwards. We release the generation and evaluation framework to support additional samples and experiments across reference depths and action models. During evaluation, the engine executes proposed folds or rejects invalid actions with explicit feedback, while the model remains responsible for selecting actions and searching for a solution. We evaluate multimodal language models under progressively greater tool assistance, and against a deterministic breadth-first search over the same action set. Candidate sequences are assessed by executing them and comparing the resulting folded state with the target, allowing for planar translations, rotations, and reflections. Under the low reasoning effort setting, none of the evaluated models solves a sample outside the easy subset. On that subset, the best model achieves a success rate of 20.4%, compared with 54.7% for the search baseline. Its dominant failure mode is not an exhausted query budget but the repeated proposal of states it has already visited, which terminates 48% of its attempts. These results identify a substantial gap between multimodal language models and explicit search on this origami planning task.

1 INTRODUCTION

Geometric planning requires a model to connect a description of a target with the operations needed to construct it. In origami, this involves choosing a fold line, identifying which parts of the sheet should move, and tracking how the resulting reflections change their positions and layer order. A locally valid fold may lead away from the target, so completing a sequence also requires selecting among alternatives and revising earlier decisions. We study whether multimodal language models can use visual and geometric observations to make these decisions over successive folds.

Existing origami benchmarks assess some aspects of geometric understanding, but answering questions about a folded configuration does not establish the ability to construct it. GamiBench (Spencer et al., 2025) uses visual question answering to assess folded configurations, viewpoint consistency, and impossible patterns. OrigamiSpace (Xu et al., 2025) includes spatial reasoning tasks and crease-pattern code generation with an interactive environment. Generating a pattern specification differs from constructing a sequence of folds that realizes a supplied target. CP2Seq evaluates whether a model can translate geometric information into a complete, executable folding sequence, with each action checked for validity and the resulting construction checked against the target. Given a crease pattern and target folded state, the model must select folds, track their effects, and revise its choices until execution reproduces both the crease pattern and the target state.

We introduce **CP2Seq**, a benchmark of 600 procedurally generated origami samples and a framework for generating, executing, and evaluating folding sequences. Each task supplies a crease pattern and a target folded state; the reference sequence and intermediate reference states are withheld. Starting from an unfolded square, the model must produce a valid sequence of simple folds under the specified action model. In the visual setting, it receives images of both inputs and views of its current folded state. The crease pattern constrains which creases must be produced, while the target specifies the arrangement of the folded sheet, including layer order. Together, these inputs guide the model’s choice of where to fold and which layers to move.

The task combines *geometric reasoning*, *spatial state tracking*, and *sequential planning*. A fold line is specified in the current folded coordinates, whereas the target crease pattern describes the original sheet. Connecting these representations requires tracking reflections and the correspondence between paper regions. Selecting a fold also requires reasoning about overlapping layers and their connectivity. Subsequent decisions depend on the state produced by earlier actions, so increasing sequence depth extends the planning horizon. We evaluate these abilities through the construction of valid folding sequences across increasing reference depths. The environment represents flat-folded states with explicit layer ordering.

We generate examples procedurally by applying legal folds and recording the resulting sequence, crease pattern, and folded state. This supplies reference solutions without manual annotation and lets us vary reference depth and the permitted folding operations. Releasing the generator also enables fresh evaluation instances beyond the published corpus, reducing reliance on fixed examples that future models may encounter during training (Section 3).

We also provide an evaluation framework that presents images of the input crease pattern, the target folded state, and the current state after each accepted edit. These observations let the model compare its progress with the target and choose its next action. Tools support adding and removing folds, inspecting geometry, and returning to a specified earlier step to explore another sequence. The engine checks proposed edits and returns feedback, while the model directs the search. Different levels of tool assistance allow us to study how this support affects performance (Section 6).

Evaluation checks both individual actions and the completed construction. Candidate sequences are replayed and assessed for crease-pattern agreement and agreement with the target folded state. A sequence need not match the recorded actions to be correct. Folded-state comparison accounts for a global translation, rotation, or turnover of the completed object, with the corresponding changes to layer order and face orientation, while preserving the identity of each paper region (Section 5). We also provide a deterministic breadth-first search baseline using the same candidate-fold enumerator and terminal-state comparator. This baseline measures how much can be achieved by systematic search with the available filtering, providing context for model performance beyond comparisons between language models.

Our contributions are:

1. **A dataset and generation framework.** CP2Seq contains 600 samples with crease patterns, target states, recorded fold sequences, and intermediate states. Configurable generation supports experiments across reference depths and action models, with coupling recorded to characterize geometric structure.
2. **An interactive evaluation and verification pipeline.** The framework connects visual and geometric inputs to tools for constructing and revising sequences, step-level legality checks, and final replay-based scoring. It includes corpus checks and a folded-state comparator tested against deliberately corrupted layer arrangements. It defines four assistance conditions; the current experiments cover the filtered and no-tools conditions.
3. **A search baseline and an empirical analysis of model behavior.** We provide breadth-first search over the same candidate actions, report performance by difficulty stratum, and analyze episode termination. The current evaluation concentrates on `gpt-5.6-luna`; other model runs have insufficient coverage for a cross-model ranking.

In the current aggregate results, breadth-first search solves 54.7% of easy-stratum attempts, compared with 20.4% for `gpt-5.6-luna` with filtering and 6.2% without tools. These rates summarize different sets of attempts rather than a paired comparison on an identical sample set. No evaluated language-model attempt succeeds on the medium or hard strata. All recorded reasoning-

effort settings are low, so these results do not establish a general limit on the evaluated models. State cycling terminates 48.1% of Luna’s attempts, motivating further experiments on state tracking and backtracking. Section 8 reports the coverage, budgets, and limitations of these measurements. The findings motivate evaluating geometric planning through both the constructions models produce and the search behavior that leads to them.

2 RELATED WORK

2.1 ORIGAMI BENCHMARKS AND GENERATION

GamiBench (Spencer et al., 2025) evaluates multimodal models through visual questions about folded configurations, viewpoints, and impossible crease patterns. Its viewpoint-consistency and impossible-fold-selection metrics assess whether predictions remain consistent across views and whether models recognize infeasible patterns. OrigamiSpace (Xu et al., 2025) evaluates pattern prediction, multi-step spatial reasoning, spatial relationships, and crease-pattern code generation. It also provides an interactive environment for the code-generation task. These benchmarks connect origami geometry to visual reasoning and generation, with different requirements on the model’s output.

CP2Seq focuses on recovering a complete folding sequence from a crease pattern and a target folded state. The model selects and revises actions, and evaluation checks their execution and the resulting construction. This differs from selecting a folded configuration or generating a pattern specification. Our procedural generator records reference sequences as folds are performed, allowing new instances to be produced without manually annotating solutions. The environment checks proposed actions under its supported folding model and supplies feedback during the search. Rejecting an invalid action in a reachable instance is distinct from identifying an impossible crease pattern: the former concerns a particular move, while the latter concerns the feasibility of the pattern itself.

2.2 COMPUTATIONAL ORIGAMI AND SEQUENCE PLANNING

Our action models build on the simple-fold framework of Arkin et al. (2004), which distinguishes one-layer, some-layers, and all-layers folds. CP2Seq uses the latter two variants. Work on flat-folded states studies layer ordering as well as planar geometry (Bern & Hayes, 1996; Akitaya et al., 2024), motivating our explicit representation of layers in the state and final comparison. Earlier work also addresses generating folding sequences from crease patterns (Akitaya et al., 2013); sequence recovery is therefore an established computational origami problem. We use it to evaluate multimodal models through visual observations, executable actions, and feedback. Complexity results depend on the permitted folds and crease-pattern restrictions (Arkin et al., 2004); we assess the practical difficulty of our generated instances through measured model and search-baseline performance.

3 CP2SEQ: TASK AND DATASET

3.1 TASK AND ACTION MODELS

Given a crease pattern C and a target folded state S^* , the model must construct a sequence of legal folds $a_1, \dots, a_T$ starting from an unfolded square. Replaying the sequence must reproduce the crease pattern and reach the target state under the equivalence defined in Section 5. Each state records paper geometry and layer ordering; the reference sequence and its intermediate states are withheld, and alternative valid sequences are accepted. Figure 1 summarizes the task.

We use two simple-fold action models (Arkin et al., 2004): all-layers folds move the paper on the selected side of a fold line across the full stack, while some-layers folds select a contiguous run from the top or bottom of the stack. Both use flat 180° folds, with connectivity checks to reject moves that would tear the sheet; state representations and action constraints are detailed in Appendix A.

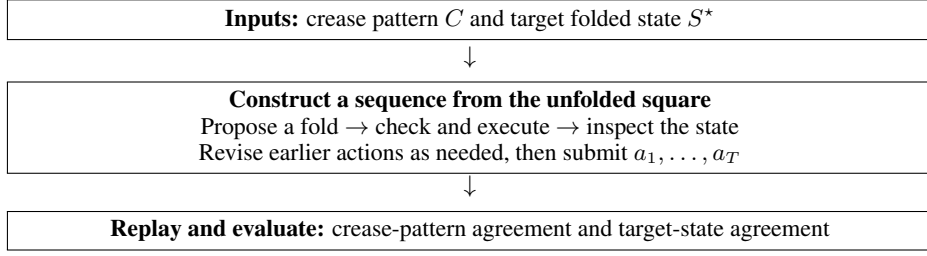


Figure 1: CP2Seq task and interactive evaluation. The model receives both endpoints and constructs the intervening fold sequence. Observations and tools depend on the assistance condition; the reference sequence and intermediate reference states are withheld.

3.2 DATASET CONSTRUCTION

We generate each example by applying a sequence of legal folds to a square sheet and recording its crease pattern, intermediate states, and final folded state. We attempt to vary search difficulty by increasing the number of folds in the reference sequence: longer sequences can require exploring more combinations of actions. Reference length is not necessarily the shortest solution length, so difficulty is assessed through solver performance. We also record coupling, the number of creases created by a fold, to characterize the geometry of each instance.

3.3 DATASET STATISTICS

CP2Seq contains 600 samples: 400 all-layers examples grouped by reference depth and 200 some-layers examples with three to six reference folds (Table 1). These counts describe distinct dataset samples, not evaluation attempts.

Table 1: Dataset composition. Fold counts refer to recorded reference sequences.

Action model	Stratum or group	Samples	Reference folds
All-layers	Easy	200	4–10
All-layers	Medium	100	11–13
All-layers	Hard	100	14–19
Some-layers	Depth 3	50	3
Some-layers	Depth 4	50	4
Some-layers	Depth 5	50	5
Some-layers	Depth 6	50	6
Total		600	3–19

Each sample contains the task inputs, a reference sequence, and metadata (Table 2). The model receives the crease pattern and target state; reference actions and intermediate reference states are withheld. Tool availability and target feedback are configured separately from dataset groups (Section 7.1).

Table 2: Contents of a CP2Seq sample.

File	Contents
<code>cp.fold</code>	Input crease pattern.
<code>steps.fold</code>	Folded states after each reference action; only the final state is supplied as the target.
<code>seq.json</code>	Withheld reference actions: fold lines, directions, layer selections, and creases created.
<code>meta.json</code>	Sample descriptors and generator version.

Reference sequences are checked by replaying them and comparing the resulting creases with the stored pattern. The two corpus checks reported in this release pass on all 600 samples; they share the folding engine and therefore do not independently validate its physical assumptions.

Coverage is limited in two respects: long reference sequences with low coupling are sparsely represented by the generation procedure, and some-layers examples extend only to six reference folds. These are limits of the present corpus, not proofs that other configurations are impossible.

4 THE ENVIRONMENT, WHICH IS ALSO THE VERIFIER

4.1 ONE OBJECT, TWO NAMES

In this benchmark the environment and the verifier are the same object. Stepping it is the same act as asking for a verdict: the model submits a candidate fold, and the reply is either the resulting state or a refusal. There is no separate validity oracle, no second artifact that could disagree with the first, and no learned component in the loop.

4.2 THE DIVISION OF LABOUR

The implementation provides sequence-editing tools (`add_fold`, `remove_fold`, `go_to_step`, and `restore_revision`), observation tools (`get_state` and `get_images`), optional candidate filtering (`list_legal_folds`), optional target feedback (`compare_to_target`), and submission (`finish`). Which tools are exposed is a run configuration; target feedback has three separately configured levels (Section 7.1).

Three verification roles must be distinguished. The step engine checks proposed actions and target-crease compatibility. Final replay compares the constructed pattern and folded state with the target. A separate Flat-Folder wrapper checks whether a crease pattern admits a flat-folded state using layer-order constraints, including taco-taco, taco-tortilla, tortilla-tortilla, and transitivity constraints (Akitaya et al., 2024). This last check does not verify the proposed sequence or establish that its particular layer ordering is valid, and is not automatically invoked by every model action. These checks should not be described collectively as a theorem that guarantees physical realizability.

Table 3: Who does what. The environment does not search.

COMPONENT	RESPONSIBILITY
Environment / verifier	State update and legality. It does not search and does not choose
Model	Proposes the next fold, decides where to go, recognises when to backtrack

The environment checks and executes actions; it does not select a path to the target. Optional candidate filtering removes illegal or off-pattern actions, while the model chooses among the remaining alternatives. The search baseline measures what systematic exploration can achieve with the same candidate enumerator.

4.3 REFUSALS ARE NAMED, ISN'T THAT YOU SHOULD MENTION A CHECK, EACH STEP CHECK AND FINAL STEP CHECK, FINAL STEP CHECK NOW IS MISSING, SHOULD MENTION HERE———

An illegal fold returns a structured refusal with a name, not a boolean.

The names are the design. A model told `would-tear` can act on the information; a model told “illegal” cannot. This also turns error analysis into a taxonomy that exists before the experiment: every rejection arrives already labelled with the class it violated, so a reject is a diagnosis rather than a dead end.

Final step check. - The four constraint classes that Flat-Folder checks (taco-taco, taco-tortilla, tortilla-tortilla, transitivity) (Akitaya et al., 2024; Flat-Folder) remain the right vocabulary for global terminal states; they are not what a single legal step needs checking against here.

Table 4: The refusal taxonomy. A rejection arrives already labelled with the class it violated.

REFUSAL	MEANING
would-tear	The moving layers are joined to stationary paper away from the fold line, so the sheet would come apart. The binding constraint of the some-layers action model
no-crease	The fold moves layers without creasing anything, tearing them free of the sheet rather than folding them
nothing-to-move	The fold line misses the selected layers entirely
direction-impossible	A run taken from the bottom of the stack was asked to fold over, or the reverse
bad-line	The fold line is malformed, or names no side of itself

4.4 WHAT THE MODEL OBSERVES - YOU SHOULD ADD A SCREEN SHOT OF SIMULATOR SOFTWARE !!!!!!!!!

On success the environment returns the new state together with two views: a top-down X-ray of the stack, and an exploded view of the layers. Every intermediate state in this tier is flat, so a second camera angle would show the same silhouette rotated and would carry no new information; what carries information is layer structure, which is why the second view is an explosion rather than a rotation. Positions are reported as coordinates on the original sheet together with an integer layer index. There is no continuous vertical coordinate at this tier, and a rendering that looks three-dimensional is not evidence of one; thickness belongs to a different problem.

5 SCORING

5.1 WHY THE OBVIOUS PROTOCOL IS WRONG

The recorded sequence is *a* solution, not *the* solution. The generator folds; it does not search, so nothing makes its sequence minimal, and no minimal-length label is shipped because producing one would require a search this benchmark deliberately does not run. Measured on the verified split, 2 of 100 samples admit a sequence one fold shorter than the one that built them.

The consequence is sharp. **Any metric that compares a proposal step by step against the recorded sequence marks a better answer wrong.** A model that finds a shorter correct solution is penalised for it. This disqualifies edit distance, and it disqualifies step-level precision, recall and F1 against an expert sequence for this task. Step count is not a correctness signal in either direction.

5.2 WHAT WE DO INSTEAD

Proposals are scored by replaying them through the engine and comparing the resulting states. Both sides are replayed: the model’s sequence and the reference sequence both pass through the same engine, and a stored frame is never the object compared against. That last point is a consequence of the file format rather than a preference. Stored frames record where each face sits in the current plane and not which piece of the original sheet it is, and deep in a stack many layers are congruent triangles, so two states differing by a swap of two such layers cannot be distinguished from stored frames alone. A replayed state carries original-sheet coordinates for free, because the engine needs them to decide tearing at all.

Two levels of equality are reported.

Level 1, crease sets. An exact multiset comparison of maximal creased intervals, equal rather than overlapping, with no tolerance in the verdict. This level is genuinely tolerance-free: edges are clustered into lines first, so subdivision differences cannot register as disagreements, and what remains is a combinatorial comparison.

Level 2, folded states up to a declared group of transforms. Because one crease pattern admits many valid terminal states, exact equality against the one stored state would mark correct answers

wrong. The answer is an equivalence: declare in advance which differences do not count, then demand equality inside that.

The transforms are quotiented out of every result reported in this paper. The declared group is the full plane isometry group: **arbitrary translation, arbitrary rotation, and reflection**. One single isometry must carry every layer of the candidate onto its reference counterpart; a match is not accepted by transforming each layer independently.

Each transform is in the group for a concrete reason rather than for generality. *Translation*, because a fold can carry paper off the original square and which half of the sheet travels decides where the stack lands; a folded model is the same model wherever it sits on the table. *Rotation*, because the same object produced by a sequence that worked around the square in a different order arrives rotated; rotations preserve both stack order and face parity, so they need no further correction. *Reflection, with two corrections*, because a reflection represents turning the model over, and turning a model over mirrors the coordinates, **reverses the stack** and **flips every face’s parity** at once. All three are applied together. Applying a reflection to coordinates alone would compare a model against its mirror image and silently accept wrong answers, which is the most dangerous failure a comparator of this kind can have.

Floating point is unavoidable once transforms are admitted, and this paper states its tolerance rather than claiming not to have one. A rotation by an angle that is not a multiple of a right angle produces irrational coordinates, so candidate and reference vertices agree only to within floating-point representation, and comparing them for bitwise equality would reject correct answers. The comparator matches polygons under a tolerance of 2×10^{-6} in the plane, with degenerate-vertex cleanup at 10^{-7} and a collinearity threshold of 10^{-9} . These values are reported in every result record, so a reader can see what was used rather than infer it.

Caveat. This qualifies a claim made elsewhere. Level 1 crease-set comparison is genuinely tolerance-free, since the verdict involves no floating-point threshold. Level 2 is not, and cannot be, because it admits rotations. The precise formulation is that the *verifier* is exact, since legality is decided combinatorially by the engine, while *terminal-state equality under transforms* carries a stated numerical tolerance.

Caveat. The layer-order variants that a flat-foldability solver reports as equally valid are deliberately *not* in the group. Those are not symmetries of one state; they are different states reachable from the same crease pattern, and deciding which orderings are valid requires a solver. Folding that into an equality test would hide an external dependency inside what reads as arithmetic. If that equivalence is wanted, it belongs in a separate check that owns the solver.

5.3 METRICS

The primary metric is **solve rate at a fixed query budget**, reported per difficulty stratum and never pooled, under both levels of equality. A proposal is solved if replaying it reproduces the target. Exhausting the budget is a timeout, which is reported separately from a wrong answer because the two mean different things.

The secondary metric is **queries to solution**, reported over all attempts including those that time out. Conditioning on the solved attempts hides precisely the long-tail blowup that makes the task interesting. This metric is what answers the “the simulator did all the work” objection quantitatively: two models sharing one exact verifier can be ranked by how little of it they needed.

5.4 VALIDATING THE PROTOCOL

A verifier paper that does not test its own verifier is asking to be taken on trust, so the comparator ships with a negative control: a deliberately corrupted state, with two layers swapped, must be rejected.

The control earned its place. On its first full run, 23 of 600 corrupted states were **accepted**. The cause was not a tolerance. The comparison used current-plane geometry and parity alone, and deep in a stack many layers are congruent triangles, so two layers occupying the same region but originating from different parts of the sheet were interchangeable. They are not interchangeable: which one lies underneath is a real difference between two folded states, and precisely the difference

Level 2 exists to catch. With face identity on the original sheet restored to the comparison, the full corpus reports 600 states equal, all under the identity symmetry, and 600 of 600 corrupted states rejected. Every verdict additionally reports whether face identity was available, so a weakened comparison is visible in the output rather than assumed away.

6 EXPERIMENTAL SETUP

6.1 THE PIPELINE —SHOULD HAVE A HAND DRAWN GRAPH

One episode is one sample attempted by one model under one tier of assistance. The loop is the same for every arm and every model, and the only thing that varies between arms is what the model is allowed to see and call.

```
EPISODE(sample, model, tier, budget):
  cp, final <- LOAD(sample)      # cp.fold, and the LAST frame of
                                # steps.fold; other frames withheld
  state <- unit square, one layer
  seq <- []
  while len(seq) < budget:
    obs <- RENDER(state, tier)    # withheld at the no-vision tier
    action <- model(cp, final, obs, history, tier)
    if action is SUBMIT: break
    verdict <- ENVIRONMENT.step(state, action) # verifier IS this
    if verdict is REFUSAL:
      history.append(refusal with its reason) # state unchanged
    else:
      state <- verdict.state
      seq.append(action)
      history.append(obs)
  return SCORE(replay(cp, seq), replay(cp, reference_seq))
```

Three things about this loop decide what the benchmark measures.

The environment is the only thing that changes state. A model cannot assert that a fold happened. It proposes, and the environment either performs the fold or refuses it, so the state the model reasons about is always a state paper could be in.

A refusal costs a query and does not advance the episode. This is what makes queries-to-solution meaningful: a model that proposes carelessly pays for it in budget, and a model that reasons before proposing is rewarded in the secondary metric even when both eventually solve the sample.

The reference sequence is never read during the episode. It enters only at scoring time, and then only as something to replay, never as something to compare against step by step (Section 5.1).

6.2 THE TOOLS THE MODEL IS GIVEN

The tool belt is the experimental variable. Every tool is a read or a step against the same engine; none of them searches, and none of them ranks candidates.

`list_legal_folds` is the tool that needs justifying, because it is doing the most work. It enumerates candidate actions and returns those that are legal, which is a large prune performed by the simulator rather than by the model. That is precisely why the tier structure exists and why the deterministic baseline of Section 6.4 is reported: if exhaustive search over the enumerated set solves the corpus outright, then the filtered arm is not measuring origami reasoning, and the benchmark has to say so rather than let a headline number imply otherwise.

6.3 TIERS, MODELS AND PROTOCOL

Tiers. Four levels of assistance, from a full tool belt down to none; the table is in Section 7. The same model, the same samples and the same query budget across tiers, so that the only difference between two tiers is what was withheld.

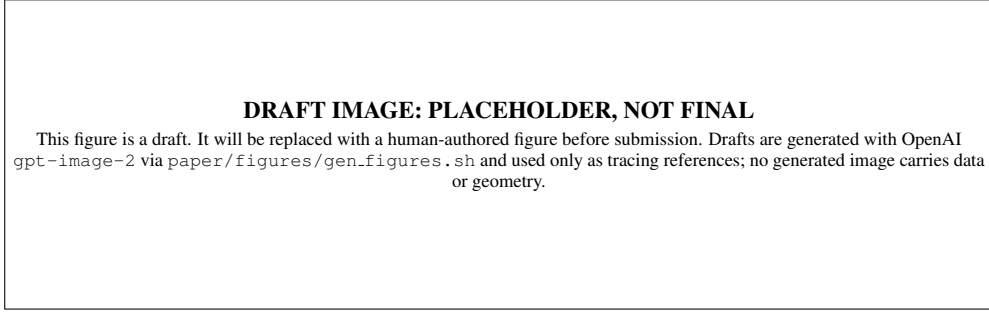


Figure 4 placeholder: the evaluation pipeline. Inputs on the left; the propose-verify-observe loop in the centre with the environment and verifier drawn as one box and refusals looping back without advancing; replay and scoring on the right, with the reference sequence outside the loop.

Figure 2: [DRAFT IMAGE] One episode. The environment and the verifier are the same object, so stepping is the same act as asking for a verdict. The reference sequence enters only at scoring.

Table 5: The tool belt. Tiers are defined by which of these are available.

TOOL	WHAT IT RETURNS	WHY IT EXISTS
add_fold	The new state, or a named refusal	The action. This call is the verification
get_images	Top-down X-ray of the stack, exploded layer view	The visual channel; withheld at the no-vision tier
get_state	Face positions in original-sheet coordinates plus integer layer index	The symbolic channel, so a model is never forced to read geometry off a raster
list_legal_folds	Every fold the engine would currently accept	The filtering tier. It prunes; it does not choose
go_to_step	A specified earlier step	Backtracking without spending the episode

Models. [TO FILL from the aggregation: the models actually run, their versions, and the reasoning-effort and image-history settings used. Held fixed across tiers, because comparing across models and across tiers at once would confound the two.]

Protocol. Query budget, number of repeats are fixed before the first run and reported with the results. Median and spread across repeats are reported, never a single run, because sampling is not deterministic and one run is not a measurement.

Transforms are applied in every reported result. Every solve rate in Section 8 is computed after quotienting out the plane isometry group of Section 5.2: translation, rotation and reflection, with reflections reversing the stack and flipping face parity. A candidate is counted correct if one single isometry carries every layer onto its reference counterpart. This is not a post-hoc leniency applied to borderline cases; it is part of the definition of a correct answer, applied identically to every arm, every model and every baseline. Turning it off would mark correct answers wrong, because where a folded model lands on the table is an accident of which half of the sheet travelled.

Matching under transforms requires a floating-point tolerance, and it is stated. Admitting arbitrary rotation means candidate and reference coordinates agree only to within floating-point representation, so the comparator matches polygons within 2×10^{-6} , cleans degenerate vertices at 10^{-7} and treats vertices as collinear below 10^{-9} . Every result record carries the tolerance it was scored under, so the value is visible in the released runs rather than buried in a constant. The engine’s

DRAFT IMAGE: PLACEHOLDER, NOT FINAL

This figure is a draft. It will be replaced with a human-authored figure before submission. Drafts are generated with OpenAI gpt-image-2 via `paper/figures/gen_figures.sh` and used only as tracing references; no generated image carries data or geometry.

Figure 5 placeholder: a refusal as the model receives it. The proposed fold line over the current state, the named reason would-tear, and the offending crease segment highlighted; beside it the same state with a legal fold line.

Figure 3: **[DRAFT IMAGE]** A refusal is a diagnosis, not a dead end. The same state and the same layer selection give opposite verdicts depending only on the orientation of the fold line.

legality decisions are combinatorial and involve no tolerance; the numerical threshold applies only to terminal-state equality under transforms.

Error verbosity is itself a variable. The richer a refusal, the more of the reasoning the environment performs rather than the model; at the limit a message that names the fix has solved the step. Verbosity is fixed before the first run and reported, otherwise the comparison between tiers measures message design rather than reasoning.

6.4 BASELINES

Three, none of them a language model, so the models are ranked against something rather than only against each other.

Table 6: The baseline ladder. [TO RUN]

BASELINE	WHAT IT ESTABLISHES
Uniform random over enumerated legal actions	The floor. What the tool alone achieves with no reasoning at all
Deterministic breadth-first over the same action set (workspace/search_baseline.mjs)	How much of the task the enumerator already solves, and the depth at which exhaustive search stops being affordable
Symbolic crease-pattern-to-sequence solver (Creasy)	The closest thing to prior art on the task itself

The breadth-first baseline is the one that answers the “the simulator did all the work” objection of Section 7 numerically rather than structurally, which is the stronger form of that answer.

7 PROGRESSIVE TIERS OF TOOL ASSISTANCE

[TO RUN]

The four conditions below describe the planned assistance study. They are distinct from the folding action models and the target-feedback levels described below. The current results do not instantiate the full study; completed conditions and coverage are reported in Section 8.

Same model, same patterns, same query budget across tiers. If the no-vision or no-tools tier performs nearly as well as the full tier, **that is a finding rather than a failed experiment**: it means the gain is not where it was assumed to be. [TO RUN]

Table 7: The tiers of assistance. Same model, same patterns, same query budget across tiers.

TIER	TOOLS AVAILABLE	WHAT IT ISOLATES
Full tool belt	Environment, rendered views, any auxiliary tools	Upper bound
No vision	Same, rendered views withheld	The contribution of visual feedback
Verifier only	Pass/fail, no filtering	The contribution of filtering on top of raw verification
No tools	None, anonymized geometry	Memorization control

7.1 TARGET-COMPARISON FEEDBACK

The optional `compare_to_target` tool has three feedback levels, configured for the run rather than selected by the model. These levels are separate from the assistance conditions above and from the two final scoring criteria in Section 5. The tool reads the current and target stacks without consulting the reference sequence or changing the state.

Table 8: Implemented target-feedback levels. Their availability does not establish an experimental benefit; comparisons require matched runs.

Level	Information returned
1: Counts	Current and target layer counts, their difference, and a count-based summary.
2: Diagnosis	Level 1 plus aligned layer matches, mismatched ranks, and a diagnosis of count, shape, orientation, or ordering disagreement.
3: Corrections	Level 2 plus suggested corrections for up to twelve mismatched ranks, such as orientation changes or a different target rank for a layer shape.

The richer levels supply more target-specific guidance. Their diagnoses are geometric feedback, not proofs of a valid repair or substitutes for the final state comparator. In particular, a matching layer count does not establish that the sequence is complete or correct.

8 RESULTS

All numbers below come from `workspace/RESULTS/results.md`, produced by `python3 workspace/aggregate_results.py` over every saved run. 1,211 attempts across 47 runs and 256 distinct samples. Every attempt is counted once, and an attempt with no recorded `solved` field is counted as unsolved.

8.1 HEADLINE

No language model solved a single sample beyond the easy stratum. Across every model arm, mid is 0 of 70 and hard is 0 of 27. On the easy stratum a deterministic breadth-first search over exactly the action set the models are given solves 54.7%, against 20.4% for the best-covered model at the same tier. **Blind search beats every model tested, by a factor of about 2.7 on the only stratum where anything succeeds at all.**

Caveat: read the denominators before the percentages. Only `gpt-5.6-luna` and the baseline have coverage worth a rate. `gpt-6-astra` shows 1/1; **that is one attempt and must not be reported as 100%.** `claude-sonnet-5`, `gpt-5.6-sol`, `gpt-5.6-terra` and `codex-cli-chatgpt` have between one and seven attempts each. These rows are present because omitting them would be selective reporting, not because they support a comparison. Any main table in the submitted paper prints n in every cell or drops the row.

Table 9: Solve rate per stratum, all saved runs. Denominators are attempts, not distinct samples. Rows with fewer than ten attempts support no rate and are shown only so that reporting is not selective.

ARM	TIER	EASY	MID	HARD	OVERALL	CALLS
deterministic-bfs	legal-folds	366/669 (54.7%)	1/40 (2.5%)	0/16	367/725 (50.6%)	5
gpt-5.6-luna	legal-folds	59/289 (20.4%)	0/66	0/25	59/380 (15.5%)	23.5
gpt-5.6-luna	no tools	4/65 (6.2%)	0/1	0/1	4/67 (6.0%)	29
claude-sonnet-5	legal-folds	0/5	–	–	0/5	10
gpt-5.6-sol	no tools	0/7	–	–	0/7	20
gpt-5.6-terra	no tools	0/1	–	–	0/1	35
gpt-6-astra	no tools	1/1	–	–	1/1	5
codex-cli-chatgpt	none	0/1	–	–	0/1	2
errored, model unrecorded	–	0/20	0/3	0/1	0/24	–

Stratum (reference folds)	Median states expanded	Solved / attempts
Easy (4–10)	65	366/669
Medium (11–13)	1,051.5	1/40
Hard (14–19)	532	0/16

Figure 4: Observed BFS search effort in the saved aggregate. Bar lengths use a linear scale and show median states expanded across attempts, including unsuccessful attempts. Budgets and repeated samples are not standardized by this aggregate; 358 of 725 attempts terminated in timeout. The lower hard-stratum median does not imply that hard instances require less search. Reference depth is not minimum solution length.

8.2 THE BASELINE READING

deterministic-bfs solves 367 of 725 attempts overall. Per stratum: 366/669 easy, 1/40 mid, 0/16 hard. Median states expanded is 65 on easy, 1051.5 on mid, 532 on hard. 358 of its 725 attempts ended in timeout rather than exhaustion, so **50.6% is a lower bound under the baseline’s time budget, not the ceiling of exhaustive search.**

This is the outcome Section 16.7 anticipated and the one `workspace/search_baseline.mjs` was written to detect. Its header records that `list_legal_folds` evaluates roughly 2,010 candidate actions per state and returns a mean of 3.51 that are legal and stay inside the target crease pattern, over 73,750 enumerations. A branching factor near 3.5 makes an easy sample of five folds a tree of a few hundred states, and the median of 65 states expanded on easy confirms it. **The easy stratum is therefore not measuring origami reasoning.** It measures whether a model can avoid losing to breadth-first search on a tree its own tool has already pruned roughly 570-fold, and every model tested loses.

We state this rather than report the models’ easy-tier rate as an achievement. It is also the answer to the “the simulator does all the work” objection of Section 7, and the answer is uncomfortable: on the easy stratum the simulator does do most of the work. The benchmark’s value here is that it measures that rather than hiding it. On mid and hard the objection dissolves, because search solves 1 of 56 and the models solve 0 of 97.

8.3 WHAT THE TOOLS CONTRIBUTE

The one within-model tier comparison the data supports is gpt-5.6-luna on easy: 59/289 (20.4%) with the filtering tool against 4/65 (6.2%) with no tools, so filtering is worth roughly 3.3× on that stratum and nothing on mid or hard, where both tiers are zero.

Caveat. The no-vision and verifier-only tiers of Section 7 were not run. The four-tier ladder this paper describes is not yet instantiated; only its two endpoints exist.

Table 10: Termination reasons for gpt-5.6-luna, 447 attempts.

TERMINATION	<i>n</i>	SHARE
state_cycling	215	48.1%
finished	167	37.4%
turn_budget	47	10.5%
repetition_detected	18	4.0%

8.4 FAILURE TAXONOMY

This is the most informative result in the section, and it is not a solve rate. A majority of attempts, 233 of 447 counting cycling and repetition together, end with the model revisiting a state it has already produced rather than exhausting its budget. The model is not running out of room to search; it is failing to notice it has been somewhere before. That is a specific, diagnosable deficit in maintaining state across a multi-step spatial task, and it is the kind of finding a benchmark with an executing verifier can produce and a multiple-choice benchmark cannot. The baseline never cycles, because breadth-first search over a visited set cannot; it times out instead, 358 times.

8.5 WHERE DIFFICULTY LIVES

Depth separates the arms completely. Easy is the only stratum where anything succeeds; mid yields a single solve, by the baseline, at a median of 1051.5 states expanded against 65 on easy; hard yields none from any arm. [TO RUN: solve rate against coupling, to test whether the second stratification axis of Section 3.2 predicts difficulty as intended. Only depth is evidenced so far.]

8.6 THREATS TO THESE RESULTS

Every model attempt ran at low reasoning effort. All 461 language-model attempts with a recorded setting used `reasoning_effort: low`; 25 have none recorded. This is the single largest caveat on every claim in this section. “Frontier models fail this task” is not supported by these runs. What is supported is that frontier models at low reasoning effort fail this task and lose to breadth-first search where search works at all. Re-running the best-covered arm at high effort is the most valuable remaining experiment, and until it exists the headline carries the qualifier.

Coverage is severely imbalanced. 725 baseline attempts and 447 for gpt-5.6-luna, against between one and seven for every other model. No cross-model claim is made.

24 attempts terminated in error with no model recorded and are counted as unsolved. They are reported as their own row rather than dropped, because dropping failed attempts inflates every rate above them.

Repeats are not yet reported as median and spread. Section 6.3 promises this; the aggregation counts attempts rather than grouping repeats per sample. The protocol requires it before submission.

DRAFT IMAGE: PLACEHOLDER, NOT FINAL

This figure is a draft. It will be replaced with a human-authored figure before submission. Drafts are generated with OpenAI gpt-image-2 via `paper/figures/gen-figures.sh` and used only as tracing references; no generated image carries data or geometry.

Figure 6 placeholder: solve rate per difficulty stratum, one line per model, with the deterministic baseline as a distinct reference line crossing above every model line on the easy stratum, and attempt counts printed at each point.

Figure 5: [DRAFT IMAGE] Main result per stratum. The baseline crosses above every model line on easy; that crossing is the most informative feature of the chart and is not smoothed away.

9 LIMITATIONS

Every model attempt ran at low reasoning effort. All 461 language-model attempts with a recorded setting used `reasoning_effort: low`. The results support a claim about models at that setting, not about frontier models in general, and the headline is worded accordingly. Raising the effort on the best-covered arm is the most valuable remaining experiment.

The easy stratum does not measure what the benchmark is for. Deterministic breadth-first search solves 54.7% of it at a median of 65 states expanded, beating every model. The stratum measures whether a model can beat trivial search on a tree its own tool has pruned roughly 570-fold. Claims about reasoning should be read off mid and hard, where every arm including search is at or near zero, and where the benchmark currently discriminates nothing either. **The band in which this benchmark separates models may be narrow or, at this depth range, empty.**

Model coverage is severely imbalanced. 447 attempts for one model and between one and seven for four others. No cross-model comparison is supported.

No training. Every arm is an off-the-shelf model driven by prompting and tool calling; no model is trained or fine-tuned. The contribution is the harness, not a model. This is stated in the introduction as well as here, because a limitation the authors declare is context and one a reviewer discovers is an objection.

Scope of the action space. 89.3% of the 366 real crease patterns in Flat-Folder’s `examples/instagram/` corpus lie outside all-layers simple folding, and pre-creasing, the operation real folders use to reach long sequences at low coupling, is excluded.

The corpus verifiers share an engine with the generator, so they cannot detect a wrong model of paper. They detect drift between what was folded and what was recorded.

Attributing failures to individual capabilities. Successful construction requires geometric reasoning, spatial state tracking, and sequential planning together. A failed attempt alone does not identify which capability failed. Isolating their contributions requires targeted experiments; changing tool availability does not necessarily isolate a single capability.

Generation parameters and difficulty. Reference sequence depth is not a guarantee of minimum solution length, and variation across generated instances does not by itself establish greater difficulty. Difficulty must also be assessed through solver performance.

Evaluation contamination. Generating fresh instances reduces reliance on a fixed public test set, but does not guarantee freedom from training overlap. Models may have encountered the generator, related folding procedures, or structurally similar examples.

Flat states only. The environment represents flat-folded states with explicit layer ordering. It excludes paper thickness, material mechanics, and continuous collision checking during folding motions. Passing the engine’s checks therefore establishes validity within the supported discrete model, not full physical realizability.

Level 2’s group is narrower than the truest notion of equality, because valid layer-order variants are excluded for the reason given in Section 5.2. It is therefore strict rather than permissive, which is the safe direction but not a free one.

10 FIGURES

11 LICENCE AND AVAILABILITY

The corpus generator, the environment and verifier, both corpus checks, the state comparator and the evaluation harness are released under the **BSD 3-Clause** licence.

Table 11: Planned figures and their status.

#	FIGURE	STATUS
1	Task schematic: inputs, sequence construction, and replay checks (Figure 1)	Included
2	Optional connectivity example for Appendix A.3; verify both actions with the engine before drawing	To verify
3	A refusal as the model receives it: view, named reason, per-constraint checklist with locations	To draw
4	The difficulty grid, depth against coupling, with the empty cell visible	To draw
5	Main result per stratum	[TO RUN]

Code and data are released on publication; links are omitted here.

The artifacts carry no personal data, no human-subject data and no scraped content. Every sample is synthesised by folding a square, so no licensing question attaches to the geometry and no attribution is owed to any origami designer. Flat-Folder’s `examples/instagram/` set, the one external corpus this paper measures against, is not redistributed; the scope measurement of Section 9 was computed against it in place and is reported as a number.

12 USE OF LARGE LANGUAGE MODELS

ICLR 2026 requires that **any** use of a large language model be disclosed, on the Code of Ethics principle that all contributions to the research must be acknowledged, and holds the authors responsible for everything in the paper regardless of what assisted in producing it. The policy does not treat a model used as a research instrument differently from one used for writing: both must be disclosed. Disclosure is required **in the paper’s text and in the submission form**, so filling in this section is not sufficient on its own.

Caveat. Remember the submission form. It is a separate field and is easy to miss.

Large language models are part of the object of study. This is a benchmark for evaluating language models, so models appear in the work by construction. Every number reported in Section 8 is the output of a frontier multimodal model driven through the harness described in Section 6. The models evaluated, their versions and their settings are listed there. No model was trained or fine-tuned for this paper.

Code. The benchmark’s implementation was written with AI assistance, principally Claude Code, used as a coding assistant throughout. Every design decision, every experiment, and the direction of the work were determined by the human authors, and all generated code was reviewed by them before it was used. The authors are responsible for the correctness of the released artifacts. Two habits in the project exist because of this reliance and are worth naming: the corpus carries two independently written checks over every sample (Section 3.3), and the state comparator ships with a negative control that must reject a deliberately corrupted state (Section 5.4). The second of these caught a real defect, described in that section, in which 23 of 600 corrupted states were initially accepted.

Literature search. The related work of Section 2 was assembled primarily by hand, using Google Scholar and alphaXiv. AI assistance was used partially, to locate candidate papers and to recover bibliographic details. Every citation in this paper was verified against the arXiv or publisher record by a human author; citations that could not be verified were removed rather than softened.

Writing. AI assistance was used in drafting and editing the manuscript. The argument, the claims, the experimental design and the conclusions are the authors’. No text was included that an author

had not read and agreed with, and no result, citation or number in this paper was produced by a language model without being checked against the artifact it describes.

Figures. Any figure marked **DRAFT IMAGE** is a placeholder generated with OpenAI gpt-image-2 and is used only as a compositional reference to be redrawn by hand. Generated images are never used to depict data or geometry: crease patterns, folded states and result charts are rendered from the corpus and from the run outputs. Draft figures will not appear in a submitted version of this paper.

ACKNOWLEDGMENTS

Withheld for anonymous review.

REFERENCES

- Naaisha Agarwal, Yihan Wu, Yichang Jian, Yikuan Hu, Nishad Mansoor, Mohan Li, Yifei Peng, Wang-Zhou Dai, Yao-Xiang Ding, and Emanuele Sansone. OrigamiBench: An interactive environment to synthesize flat-foldable origamis. *arXiv preprint arXiv:2603.13856*, 2026. VERIFIED 2026-09-21 against the arXiv record.
- Hugo A. Akitaya, Jun Mitani, Yoshihiro Kanamori, and Yukio Fukui. Generating folding sequences from crease patterns of flat-foldable origami. In *ACM SIGGRAPH Posters / Student Research Competition*, 2013.
- Hugo A. Akitaya, Erik D. Demaine, and Jason S. Ku. Computing flat-folded states. In *Origami⁸: Eighth International Meeting on Origami in Science, Mathematics and Education (OSME)*, 2024.
- Hugo A. Akitaya et al. Infinite all-layers simple foldability. *Graphs and Combinatorics*, 2019. arXiv:1901.08564.
- Esther M. Arkin, Michael A. Bender, Erik D. Demaine, Martin L. Demaine, Joseph S. B. Mitchell, Saurabh Sethia, and Steven S. Skiena. When can you fold a map? *Computational Geometry*, 29(1):23–46, 2004.
- Marshall Bern and Barry Hayes. The complexity of flat origami. In *Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pp. 175–183, 1996.
- Creasy. Creasy, 2022. Open-source crease-pattern-to-sequence implementation, Java, GPL-3.0, unmaintained since 2022. <https://github.com/xkevio/Creasy>.
- Erik D. Demaine and Joseph S. B. Mitchell. Reaching folded states of a rectangular piece of paper. In *Proceedings of the 13th Canadian Conference on Computational Geometry (CCCG)*, 2001.
- Erik D. Demaine, Satyan L. Devadoss, Joseph S. B. Mitchell, and Joseph O’Rourke. Continuous foldability of polygonal paper. In *Proceedings of the 16th Canadian Conference on Computational Geometry (CCCG)*, 2004.
- Flat-Folder. Flat-Folder, 2024. <https://github.com/origamimagiro/flat-folder>, MIT licence. Decides and enumerates flat-folded states.
- Flat Origami is Turing Complete. Flat origami is Turing complete. *arXiv preprint arXiv:2309.07932*, 2023.
- FOLD file format. The FOLD file format, 2019. <https://github.com/edemaine/fold>.
- Foldings and Meanders. Foldings and meanders. *arXiv preprint arXiv:1302.2025*, 2013.
- Graphs of Origami Flat Foldings. Realization and connectivity of the graphs of origami flat foldings. *arXiv preprint arXiv:1808.06013*, 2018.
- John E. Koehler. Folding a strip of stamps. *Journal of Combinatorial Theory*, 5:135–152, 1968.

- Layer Ordering Constraints. An algebraic approach to layer ordering constraints for origami flat-foldability. In *Origami*⁸, 2026.
- Édouard Lucas. Théorie des nombres, 1891. Attributes the stamp-folding problem to É. Lemoine; earlier literature traced by J. Touchard (1950).
- W. F. Lunnon. Multi-dimensional map folding. *The Computer Journal*, 1971.
- Jun Mitani. ORIPA: Origami pattern editor. Crease-pattern editor and folded-form estimation.
- M. Moriya et al. FoldingAgent. *arXiv preprint arXiv:2609.00377*, 2026. Reconstructs folding processes from instructional video; built on PurelandFold.
- OEIS Foundation Inc. The on-line encyclopedia of integer sequences, A000136: Number of ways of folding a strip of n labeled stamps, 2026. See also A001011, A001416.
- PurelandFold. PurelandFold, 2026. <https://huggingface.co/datasets/mayaweiz/PurelandFold>, CC-BY-4.0. 27 sequences, 337 frames.
- Random Locally Flat-Foldable Origami. On random locally flat-foldable origami. *arXiv preprint arXiv:2502.04279*, 2025.
- Simple Folding of Mixed Orthogonal CPs. Complexity of simple folding of mixed orthogonal crease patterns. *arXiv preprint arXiv:2306.00702*, 2023.
- Spatial Reasoning in MLLMs. Spatial reasoning in MLLMs: A survey. *arXiv preprint arXiv:2511.15722*, 2025.
- R. Spencer, R. Yaari, R. Vemavarapu, J. Yang, S. Ngo, and U. Sharma. GamiBench: Evaluating spatial reasoning and 2D-to-3D planning capabilities of MLLMs with origami folding tasks. *arXiv preprint arXiv:2512.22207*, 2025. VERIFIED 2026-09-21: 186 regular and 186 impossible patterns, six viewpoints, three VQA tasks, VC and IFSR metrics.
- Spin Model for Flat-Foldability. A spin model for global flat-foldability of random origami. *arXiv preprint arXiv:2403.07306*, 2024.
- Visualization-of-Thought. Mind’s eye of LLMs: Visualization-of-thought elicits spatial reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2404.03622.
- R. Xu, D. Lu, Z. Zhao, X. Tan, X. Wang, S. Yuan, J. Chen, and Y. Xu. OrigamiSpace: Benchmarking multimodal LLMs in multi-step spatial reasoning with mathematical constraints. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. VERIFIED 2026-09-21: NeurIPS 2025 poster; also arXiv:2511.18450.
- Tom Zahavy, Shaobo Hou, Thomas Tumiel, James Doran, Francesco Faccio, Xidong Feng, Alex Havrilla, Igor Khytryi, Chenglei Li, Lisa Schut, Vivek Veeriah, Arijan Abrashi, Michał Kosmulski, Robert J. Lang, Nick Robinson, Brandon Wong, Marcus Chiam, Gloria Fang, and Satinder Singh. COrigami: An AI pipeline for co-designing flat-foldable visually recognisable origami. *arXiv preprint arXiv:2606.26299*, 2026. VERIFIED 2026-09-21 against the arXiv record.

A STATE REPRESENTATION AND FOLDING CONSTRAINTS

A.1 FLAT-FOLDED STATES

A state represents the positions of paper regions in the plane, their orientations, and their layer ordering. Original-sheet coordinates identify each region and preserve its connectivity to other regions as folds change its current position. Overlapping regions can occupy the same planar area while remaining distinct layers. Their original-sheet identities therefore matter both when validating a move and when comparing the completed construction with the target. Layer ordering is also a central component of computational treatments of flat-folded states (Akitaya et al., 2024).

A.2 SUPPORTED ACTIONS

The simple-fold literature distinguishes operations by the layers they move (Arkin et al., 2004). Our all-layers action selects the paper on one side of a fold line throughout the stack. Our some-layers action restricts that selection to a contiguous run from the top or bottom. An action specifies the line in current folded coordinates, the moving side, the layer selection, and whether the moving paper folds over or under the stationary paper. The selected paper is reflected across the line, with the corresponding orientation and stack-order updates. In the implemented some-layers model, a top run folds over and a bottom run folds under.

These action models specify the permitted moves. They are distinct from the assistance conditions, which specify the observations, feedback, and tools available to the model. In particular, access to candidate enumeration changes the assistance provided without defining a new folding operation.

A.3 CONNECTIVITY AND TEARING

A move must preserve the connections between regions of the original sheet. If moving paper is attached to stationary paper along a segment away from the proposed hinge, the operation would separate connected paper and is rejected with a `would-tear` response. The check uses original-sheet connectivity together with the proposed motion; overlap in the current plane alone cannot establish whether two regions are attached. Rejection leaves the current state unchanged.

Tearing is one action constraint rather than a complete model of physical feasibility. The engine also checks the action parameters and compatibility with the target crease pattern. It represents zero-thickness flat states and does not test continuous collisions during the folding motion. Step-level legality and final target agreement are separate checks: a sequence of legal actions can still produce the wrong construction.

A.4 DEPTH AND SEARCH DIFFICULTY

The recorded sequence provides a reachable target, but need not be a shortest solution. Reference depth therefore describes generation rather than minimum planning horizon. Likewise, the number of legal moves and the probability that a random proposal is legal are different quantities: a larger proposal space can lower the latter without reducing the former. We use observed solver performance to assess difficulty rather than inferring it solely from either quantity or from worst-case complexity results for other folding variants.

B REFERENCE NOTES

The bibliography is grouped in `iclr2026_conference.bib` by the role each work plays in the argument rather than alphabetically, so that a co-author can see what is load-bearing. The notes below record what is still outstanding.

Origami benchmarks and systems for language models. Xu et al. (2025) must be cited as a preprint; `papers.md` recorded NeurIPS’25 and the arXiv record carries no venue. Spencer et al. (2025) has code and data released. Moriya et al. (2026) reconstructs folding processes from instructional video and is built on the PurelandFold dataset; code at <https://github.com/maya-moriya/FoldingAgent>. **OrigamiBench** (Agarwal et al., 2026) has a *citation missing*: it appears in our metrics survey as the source of query efficiency and geometric/semantic similarity, and its dataset is the 366patterns in Flat-Folder’s `examples/instagram/`. A full reference has to be recovered before submission. **COrigami** (Zahavy et al., 2026) has a *citation missing* as well: it appears in the metrics survey as the source of flat-foldability as a boolean and a VLM aesthetic score. Same action needed.

Complexity and theory of flat folding. Bern & Hayes (1996) is the citation under the paper’s central claim: geometry does not determine layer order, and deciding overlap order is NP hard even given a valid mountain-valley assignment. Arkin et al. (2004) defines simple foldability; the action space comes from here. Akitaya et al. (2019) covers the all-layers model, which corresponds to sheet-metal bending. Akitaya et al. (2024) is the Flat-Folder paper; deciding a global flat-folded

state is NP hard. See also Simple Folding of Mixed Orthogonal CPs, Layer Ordering Constraints and Graphs of Origami Flat Foldings. Flat Origami is Turing Complete is a defensive citation only, per Section 7. Demaine et al. (2004) separates folded state from folding motion: reachability is free, so the hard question is the discrete step structure; its predecessor is Demaine & Mitchell (2001).

The crease-pattern-to-sequence problem itself. Akitaya et al. (2013) named the problem, by the group whose tools the field now uses: reflection paths, graph rewriting, step graphs; a frog base explodes to 22,665 nodes and 30 minutes, and the authors’ own future work asks for the priority heuristic this paper’s models are being asked to supply. Creasy is an open-source implementation of the above and is usable as a symbolic baseline. [TO RUN if used]

Counting folded states. OEIS Foundation Inc. (2026) lists the foldings of a strip of stamps: 1, 2, 6, 16, 50, 144, 462, 1392, 4536, 14060, ... There is no closed form. See also A001011 and A001416, and Lucas (1891); Koehler (1968); Lunnon (1971); Foldings and Meanders; meanders and stamp foldings are the same combinatorial object.

Spatial reasoning context. Spatial Reasoning in MLLMs surveys the area. Visualization-of-Thought names the mental-imagery question this benchmark can turn into a measurement.

Tools, formats and data. Flat-Folder decides and enumerates flat-folded states and is the source of the four constraint classes cited in Section 4.3. FOLD file format is the de facto standard used by every artifact we ship. Mitani is a crease-pattern editor with folded-form estimation. PurelandFold is not used as training or evaluation data here; it is cited as the closest existing sequence-level corpus and as FoldingAgent’s data.

Statistical physics of random flat-foldability. Optional support for the claim that multiple valid states is the normal case rather than a curiosity, to be included only if Section A.1 needs reinforcement: Spin Model for Flat-Foldability and Random Locally Flat-Foldable Origami.

Deliberately not cited. Thick folding (Ku & Demaine 2016), bar-and-hinge mechanics, MER-LIN2, SWOMPS, Sim-FAST-PY, and the creased-sheet mechanics literature all belong to the thickness and mechanics line of work. They are a separate paper with a verifier several orders of magnitude more expensive, and citing them here would invite a reviewer to ask why this paper does not do that one.